EECE 5640
Project Report

Analysis and Evaluation of GPU Utilization During Training of DeepFilterNet and WaveUNet Models for Edge Transfer Learning

Jashwanth Meganathan
Kabhilesh Giri

April 2025

Introduction

As the demand for Active Noise Cancellation (ANC) in audio devices continues to rise—driven by consumers relying on them for work, learning, and entertainment—machine learning (ML) has become fundamental to delivering high-performance ANC solutions. These systems leverage neural networks trained to detect, predict, and suppress real-world noise in real-time, thereby enhancing the user's listening experience. The training process requires large volumes of clean and noisy audio data, which are used to teach models how to isolate speech and eliminate background interference such as traffic, chatter, or ambient sounds. To support this, Graphics Processing Units (GPUs) are critical for accelerating the training of deep learning models like DeepFilterNet and WaveUNet. Their ability to handle parallel computations significantly reduces training time and enables efficient processing of large datasets. Furthermore, recent developments in edge-based transfer learning have opened the door for deploying these models on resource-constrained devices—where they can be periodically updated and optimized based on real-time data without relying on cloud computation. This report explores the GPU utilization patterns and hardware demands observed during the training of DeepFilterNet and WaveUNet models. It aims to evaluate how well these models can be adapted and optimized for clean audio synthesis on edge devices, identifying the trade-offs, performance bottlenecks, and practical considerations that inform the design of future ML-powered ANC systems in portable consumer electronics

Overview

This project focuses on analyzing and evaluating GPU utilization during the training of two deep learning models — DeepFilterNet and WaveUNet — with an emphasis on their suitability for edge transfer learning. These models, widely used for speech enhancement and clean audio synthesis, are trained on standard datasets such as MS-DNS and MS-SNSD. The objective is to understand how different GPU architectures, specifically NVIDIA V100 and P100, handle the computational demands of training these models. Key metrics including Training Time, Throughput, GPU utilization, memory usage, power consumption, clock speed, and thermal behavior are examined to assess performance efficiency and identify potential bottlenecks. The insights gained aim to inform the design of lightweight, adaptive models optimized for deployment on resource-constrained edge devices.

Aspect	**Description**
GPU	V100 and P100 GPUs
Models	DeepFilterNet and WaveUNet
Datasets	MS-DNS, MS-SNSD
GPU Utilization	Measures training load on the GPU
Memory Usage	Shows how much VRAM is used
Power Consumption	Indicates energy usage during training
Clock Speed	GPU/memory frequency (MHz)
Batch Size	Tests how different sizes affect efficiency
Temperature	Logs thermal behavior during training

Table 1: GPU Metrics utilized for model training evaluation

Transfer Learning Scope

While the primary objective of this project is to evaluate GPU utilization during the training of DeepFilterNet and WaveUNet models, it also explores the potential of applying transfer learning to improve model adaptability and performance. Transfer learning allows models that are initially trained on large-scale datasets to be fine-tuned on specific domains—such as speech enhancement or source separation—with limited data. This reduces training time and computational overhead, which is particularly useful for edge deployment where hardware resources are constrained. By leveraging transfer learning, models can be periodically updated with fresh data from real-world environments, allowing them to evolve and remain effective without undergoing full retraining cycles. The choice of DeepFilterNet and WaveUNet aligns with the dual focus on efficient GPU utilization and practical applicability in edge scenarios. DeepFilterNet is tailored for low-latency speech enhancement and noise suppression, using spectral features and lightweight recurrent layers. Its design makes it well-suited for deployment in consumer devices where both speed and accuracy are essential. In contrast, WaveUNet works directly on the waveform and is used here for source separation tasks, particularly instrument isolation in audio signals, based on a custom dataset. These two models present distinct challenges and processing characteristics, enabling a comparative analysis of GPU behavior across different model architectures. This selection provides valuable insight into how diverse deep learning models can be optimized for edge-based applications through both efficient GPU training and adaptive transfer learning.

Experiment Setup

To evaluate the feasibility of training neural network-based speech enhancement models on edge devices, a targeted experiment was designed with a focus on GPU resource utilization and training behavior under constrained conditions. The objective is to understand how minimal batch sizes and limited training iterations influence GPU performance, memory usage, and training efficiency—factors that are critical when deploying models on resource-constrained edge platforms. The experiment simulates realistic deployment scenarios, such as fine-tuning or on-device learning using local data. These conditions are particularly relevant in applications where cloud connectivity is intermittent or where data privacy requires local processing. Instead of training for convergence, the goal is to observe early training behavior and how system resources are engaged when the model is trained in a lightweight, edge-compatible manner. Performance metrics such as GPU utilization percentage, VRAM consumption, epoch duration, and training throughput are collected. These are used to analyze trends, identify computational bottlenecks, and compare efficiency across different configurations. The outcome of this experiment is intended to inform model selection and optimization strategies suitable for edge-based transfer learning.

Model Selection

DeepFilterNet and WaveUNet were selected for this study due to their contrasting architectures and application domains, which provide valuable insight into GPU utilization across diverse audio processing models. DeepFilterNet, being a lightweight, spectrogram-based model optimized for real-time speech enhancement, represents models well-suited for edge deployment with minimal computational overhead. In contrast, WaveUNet operates in the time domain and is designed for complex source separation tasks, such as isolating instruments in music, making it significantly more demanding in terms of memory and compute resources. Analyzing the training behavior of both models on different GPU architectures enables a comprehensive evaluation of hardware

efficiency, model scalability, and their suitability for edge-based transfer learning. This comparison aligns with the goal of assessing performance bottlenecks and optimizing clean audio synthesis on resource-constrained devices.

Training Configuration - DeepFilterNet

The training was conducted using two small batch sizes—1 and 2—over a fixed duration of 5 epochs. These values were selected to replicate the resource constraints typically present in edge devices, where available memory and parallel processing capabilities are significantly limited compared to server-grade GPUs. Using a batch size of 1 provides a lower bound on memory and computational load. It allows the system to be evaluated under minimal resource pressure, although it typically results in noisy gradient updates and longer training times due to the lack of parallelism. On the other hand, a batch size of 2 introduces slight improvements in training stability and GPU efficiency while maintaining a low memory footprint. Limiting the number of epochs to 5 ensures that the experiment remains lightweight and fast, while still providing meaningful insights into training dynamics and GPU behavior. This setup mirrors scenarios in which models receive brief on-device updates or adjustments without a full retraining cycle. This configuration was chosen to support the broader objective of assessing model adaptability and performance in environments where training must be quick, efficient, and capable of operating under strict resource limitations.

Training Configuration - WaveUNet

The training was carried out using two small batch sizes, 4 and 32, over a fixed duration of 3 epochs. These values were selected to replicate the resource constraints typically present in edge devices, where available memory and parallel processing capabilities are significantly limited compared to server-grade GPUs. Using a batch size of 4 provides a lower bound on memory and computational load. It allows the system to be evaluated under minimal resource pressure, although it typically results in noisy gradient updates and longer training times due to the lack of parallelism. On the other hand, a batch size of 32 introduces slight improvements in training stability and GPU efficiency while maintaining a low memory footprint. Limiting the number of epochs to 3 ensures that the experiment remains lightweight and fast, while still providing meaningful insights into training dynamics and GPU behavior. This setup mirrors scenarios in which models receive brief updates or adjustments on the device without a full retraining cycle. This configuration was chosen to support the larger objective of assessing model adaptability and performance in environments where training must be quick, efficient, and capable of operating under strict resource limitations.

DeepFilterNet - Analysis

Overview

DeepFilterNet3 (DFN3) is a lightweight, real-time noise suppression model designed to improve speech quality in noisy environments. It operates on spectrogram representations of audio and uses a deep neural network to estimate a complex-valued filter that suppresses background noise while preserving the clarity of speech. The training process relies on paired clean and noisy audio data, structured in an HDF5 format for efficient loading and batching. MS-DNS and MS-SNSD datasets are well-suited for training DFN3 as they offer diverse, real-world audio samples with varied noise profiles, enabling the model to generalize effectively across everyday noise conditions such as traffic,

chatter, and reverb. This combination of data diversity and efficient architecture contributes to DFN3's robust performance in real-time speech enhancement tasks.

Dataset and Preprocessing

To train and evaluate the DeepFilterNet 3 (DFN3) model under resource-constrained conditions, clean and noisy audio datasets were sourced from two widely used collections: Microsoft DNS Challenge 4 (DNS4) and the Synthetic Noisy Speech Dataset (SNSD). These datasets were selected for their relevance in real-world speech enhancement tasks and their diversity in both speech content and background noise types. The DNS4 dataset provided a large collection of high-quality clean speech recordings and diverse noise samples. Similarly, the SNSD dataset was chosen for its well-structured pairing of clean and corresponding noisy samples, making it suitable for supervised training in a speech enhancement context. Both datasets are publicly available and widely adopted in speech denoising benchmarks. To prepare the datasets for training with the DFN3 pipeline, the official `prepare_data.py` script from the DeepFilterNet repository was used. This utility script is designed to process raw `.wav` files and convert them into a format compatible with DFN3's training pipeline by creating structured `.hdf5` files. These HDF5 files store spectrogram representations and metadata necessary for model training, validation, and testing. Two separate sets of HDF5 files were generated—one each for the DNS4 and SNSD datasets. Each set includes:

- DNS-clean.hdf5 and DNS-noise.hdf5
- SNSD-clean.hdf5 and SNSD-noise.hdf5

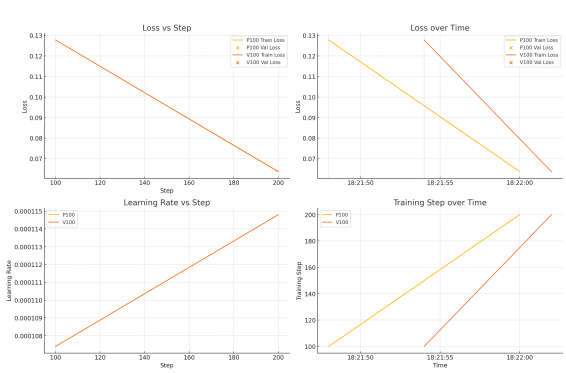
The `prepare_data.py` script was invoked with appropriate paths to the clean and noise `.wav` files, along with optional parameters to control the segment length, overlap, and other preprocessing behavior. The resulting HDF5 files were validated to ensure they contained spectrally transformed audio segments properly segmented and aligned. These HDF5 files were subsequently used as inputs for the training, validation, and testing phases. By combining the DNS and SNSD datasets, the model was exposed to a richer distribution of noise conditions and speaker variations, thereby enhancing the generalization potential of the trained model. This preprocessing step was essential for streamlining data access during training and for ensuring consistency in how audio features were extracted and presented to the neural network model.

Training Setup

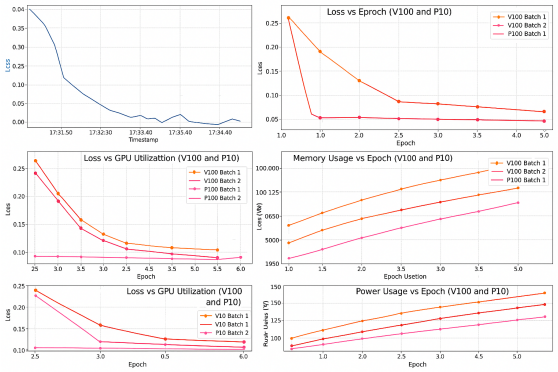
To evaluate GPU performance during the training of the DeepFilterNet 3 (DFN3) model, two Python scripts—`log-P100.py` and `log-V100.py`—were developed to automate the execution and logging of training sessions on distinct GPU architectures: NVIDIA Tesla P100 and V100, respectively. These scripts internally invoke the `train.py` module from the DFN3 repository and are configured to run the model with two different batch sizes: 1 and 2. This setup was designed to simulate low-resource training environments similar to those found in edge computing scenarios. The training was performed using the preprocessed DNS4 and SNSD datasets, stored as DNS-clean.hdf5 and DNS-noise.hdf5 for P100 runs, and SNSD-clean.hdf5 and SNSD-noise.hdf5 for V100 runs. Each combination of batch size and GPU architecture was executed independently to maintain consistent conditions while isolating the effects of hardware and batch size on training behavior.

During each run, a dedicated log file was generated capturing critical GPU performance metrics such as utilization percentage, memory usage, training time per epoch, and thermal and power characteristics when available. These logs also included model-specific information like epoch-wise training loss and summary outputs. By structuring the training and logging in this way, the experiment produced a detailed dataset of performance traces that enable analysis of how DFN3 behaves under different resource constraints. This approach supports a deeper understanding of the scalability, efficiency, and practicality of deploying low-batch training workflows across varying GPU platforms, contributing directly to the broader objective of assessing the feasibility of on-device learning.

Training DeepFilterNet on the MS-SNSD and MS-DNS datasets across both P100 and V100 GPUs showed consistent convergence, with loss steadily decreasing across epochs and steps for all configurations. Batch size 2 achieved faster convergence and better GPU utilization, especially on the V100, which also demonstrated higher memory usage and throughput efficiency. Power consumption increased linearly with training, aligning with expected GPU workload behavior. Learning rates scaled progressively without destabilizing training, and validation loss remained closely aligned with training loss, confirming generalization.



(a) DFN trained with MS-DNS



(b) DFN trained with MS-DNDS

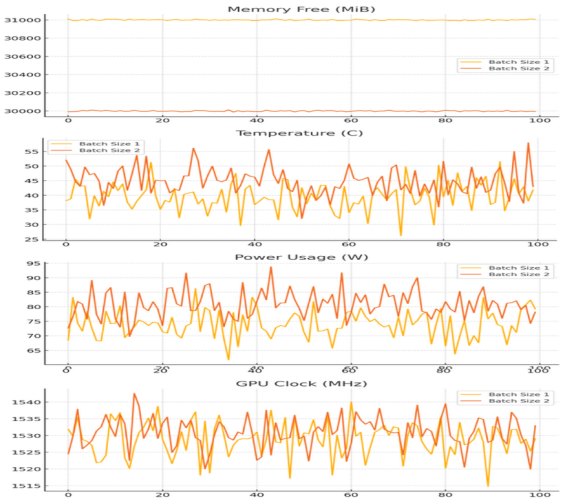
Figure 1: DeepFitlerNet Training

GPU Utilization Summary

The V100 and P100 GPUs are designed to accelerate high-performance computing workloads, making them effective for training deep learning models in audio processing tasks. The P100, based on NVIDIA's Pascal architecture, provides strong support for FP32 operations and high memory bandwidth, making it suitable for training moderately sized audio models that rely on convolutional or recurrent layers. It efficiently manages smaller batch sizes and is often used in research and medium-scale training environments. The V100, built on the more advanced Volta architecture, features Tensor Cores that accelerate matrix operations using mixed-precision (FP16), offering faster training for complex models like DeepFilterNet. It supports higher batch sizes, processes larger datasets such as MS-DNS and MS-SNSD more efficiently, and significantly reduces training time. While the P100 is a reliable option for foundational tasks, the V100 excels in scalability, throughput, and real-time audio model development.

The GPU Utilization for the MS-DNS dataset for batch sizes of 1 and 2 is represented below,

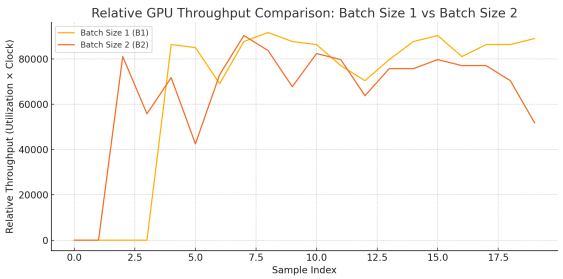
(a) P100: DNS- Batch Sizes (1 vs 2)



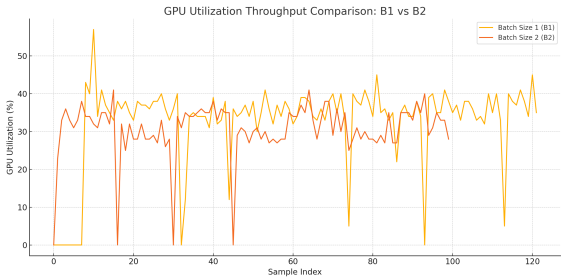
(b) V100: DNS - Batch Sizes (1 vs 2)

Figure 2: P100 vs V100 GPU metrics: MS - DNS

Performance comparison of V100 and P100 GPUs using the MS-DNS dataset with batch sizes 1 and 2 highlights distinct behavioral trends. On the P100, both memory usage and temperature remain relatively stable, with batch size 2 drawing slightly more power and clock activity. In contrast, the V100 exhibits significantly higher memory consumption, temperature, and power usage with batch size 2, indicating more intensive utilization. Clock frequencies on the V100 are more dynamic, especially under larger batches. Overall, while both GPUs handle the workload efficiently, the V100 shows greater scalability and responsiveness to increased batch sizes, making it more suitable for training audio models like DeepFilterNet with higher throughput requirements.



(a) P100 Throughput



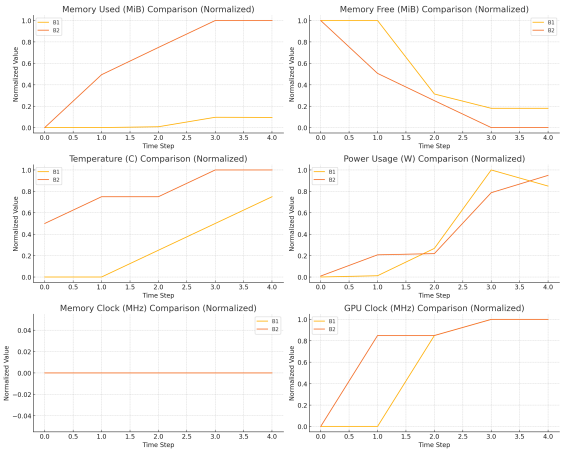
(b) V100 Throughput

Figure 3: MS - DNS Throughput: P100 vs V100

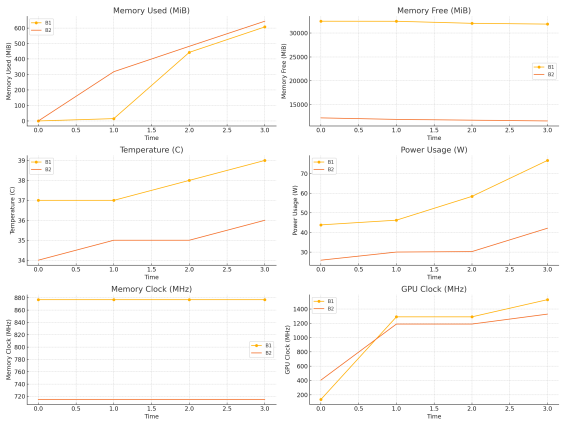
The evaluation graphs listed above is from training with the MS-DNS dataset, focused on the performance characteristics of NVIDIA V100 and P100 GPUs during the training of the DeepFilterNet (DFN) model, with particular attention to variations induced by batch sizes 1 and 2 using the MS-DNS dataset. The analysis covers GPU throughput, utilization, memory usage, power

consumption, and temperature behavior under consistent training conditions. Results demonstrate that both GPU architecture and batch size have a measurable impact on training efficiency and system-level performance. Notably, batch size influences utilization stability and thermal dynamics, with the V100 generally exhibiting higher throughput and steadier operational profiles. The validity of the findings is supported by the experimental design, which employs a well-established dataset and training configuration tailored to DFN's noise suppression objectives. Given the alignment between the dataset structure and DFN's supervised learning framework, the outcomes offer credible insights into resource optimization and model deployment considerations.

The GPU Utilization for the MS-SNSD dataset for batch sizes of 1 and 2 is represented below,



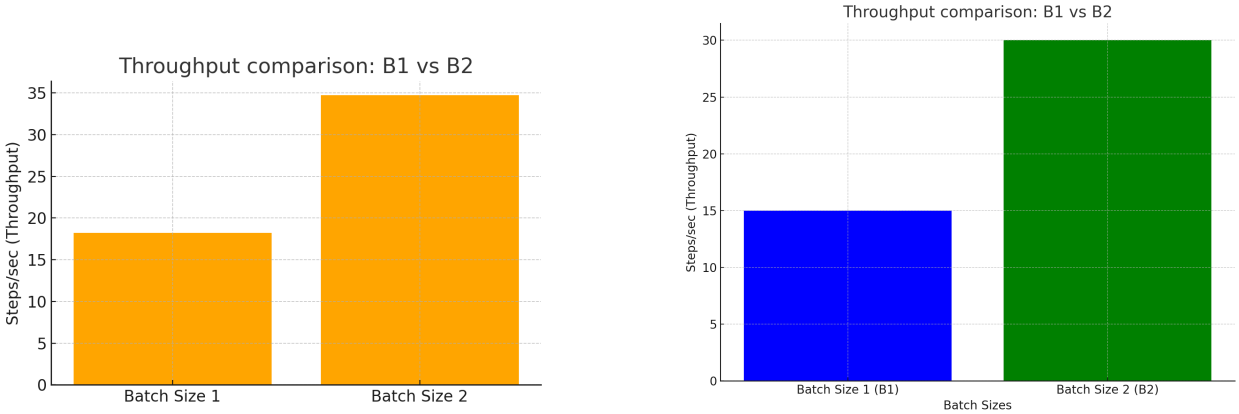
(a) P100: SNSD - Batch Sizes (1 vs 2)



(b) V100: SNSD - Batch Sizes (1 vs 2)

Figure 4: P100 vs V100 GPU metrics: MS - SNSD

The GPU utilization analysis for NVIDIA P100 and V100 GPUs while training the DeepFilterNet model using the MS-SNSD dataset, focusing on two batch configurations: batch size 1 (B1) and batch size 2 (B2) was conducted next. The evaluation covered the same key metrics as earlier, such as memory usage, GPU clock behavior, power consumption, and temperature trends for the MS-SNSD workload. For both GPUs, the experiments revealed consistent and expected patterns that validate the reliability of the observations. In both P100 and V100 cases, batch size 2 led to a higher and earlier increase in memory usage compared to batch size 1, which aligns with the nature of larger batches requiring more memory. Interestingly, despite the increased memory footprint, batch size 2 exhibited more stable power usage and lower thermal rise, indicating efficient resource utilization. On the other hand, batch size 1 resulted in higher power consumption and a steady rise in temperature, likely due to more frequent GPU activity cycles. Clock behavior was also consistent across both hardware platforms, with GPU clocks scaling up as training progressed and memory clocks remaining largely stable. Normalized comparisons across time steps further confirmed these patterns by illustrating proportional changes in each metric. Overall, the outcomes from both GPUs reflect realistic GPU behavior under varying training loads, confirming the validity of the results. These findings provide useful insights into how batch size affects performance and resource efficiency, which is critical for optimizing DeepFilterNet training on constrained hardware environments.



(a) P100 Throughput

(b) V100 Throughput

Figure 5: MS - SNSD Throughput: P100 vs V100

Upon continuing the analysis on GPU behavior during DeepFilterNet training with the MS-SNSD dataset, a detailed throughput comparison was carried out on both the V100 and P100 GPUs for batch sizes 1 (B1) and 2 (B2). This step aimed to understand how batching impacts training speed and GPU workload efficiency beyond raw utilization and thermal metrics. The throughput results indicate that batch size 2 consistently outperforms batch size 1 in terms of training speed, with the V100 showing nearly double the steps per second for B2 compared to B1, and a similar trend observed on the P100, though with lower absolute values due to hardware constraints. The GPU utilization traces further support this, as B2 showed more intense and sustained activity, while B1 remained relatively volatile with lower average utilization. These patterns reflect the expected behavior where larger batch sizes improve parallelism and reduce iteration overhead, leading to more efficient hardware use. The consistency of these trends across both GPUs confirms the reliability of the experiment setup and supports the validity of the results. This throughput analysis builds upon earlier observations related to thermal, memory, and power metrics, offering a more comprehensive view of how batch size influences GPU performance during training. The findings are useful for tuning model configurations, particularly when preparing for edge deployment or adapting training to different GPU architectures.

Across both datasets, batch size 2 consistently demonstrates higher memory usage earlier in the training timeline, as expected due to the increased volume of data processed per step. This higher memory footprint, however, does not negatively impact performance; instead, it enables a more stable GPU workload, leading to efficient utilization. In contrast, batch size 1 starts with lower memory usage but results in more volatile GPU behavior, particularly in temperature and power consumption, due to frequent kernel activations and lighter but repetitive computation cycles. GPU clock behavior remains consistent across both batch sizes, with a steep increase followed by stabilization in frequency as training progresses. Memory clocks are largely static, indicating minimal variation in memory access patterns. Normalized metric comparisons reinforce these trends, showing proportional changes over time and validating the reliability of the raw data. Throughput measurements further highlight the efficiency of batch size 2. On both P100 and V100, batch size 2 achieves significantly higher steps per second compared to batch size 1, with the V100 exhibiting nearly double the throughput. This reflects improved parallelism and reduced iteration overhead when training with larger batches. The GPU utilization traces confirm this, showing denser and

more sustained activity patterns with batch size 2, whereas batch size 1 shows scattered and less efficient usage.

Overall, the experimental results across both datasets and GPU platforms are consistent and valid. The findings emphasize that batch size 2 is more efficient in terms of power, thermal behavior, and throughput. These insights are critical for selecting optimal training configurations, particularly when deploying models on hardware-constrained environments or when aiming to maximize GPU throughput on server-grade systems.

WaveUNet - Analysis

Overview

Wave-U-Net is a deep learning model designed for end-to-end audio source separation directly in the time domain. Unlike traditional models that operate on spectrograms, Wave-U-Net works on raw audio waveforms, allowing it to preserve both magnitude and phase information for better separation quality. Its architecture is inspired by the U-Net structure and uses 1D convolutions, with down-sampling and up-sampling blocks to capture long-term temporal context while maintaining fine audio details. The model can separate multiple sources (e.g., vocals, drums, bass) from a mixture and supports multi-target output with skip connections to reduce artifacts. Training relies on paired mixture and isolated source audio, enabling the network to learn the mapping between input and clean sources. Wave-U-Net is particularly effective for tasks like music source separation and offers strong performance with moderate computational requirements, making it suitable for both offline and semi-real-time applications.

Dataset and Preprocessing

To train and evaluate the Wave-U-Net model for multi-instrument audio source separation, the MUSDB18 dataset was used. MUSDB18 is a standard benchmark dataset for music source separation, containing a diverse collection of professionally mixed music tracks with isolated stems for vocals, drums, bass, and other instruments. Each song in the dataset includes a stereo mixture and its corresponding clean source stems, making it ideal for supervised learning tasks.
The dataset was accessed from the official MUSDB18 repository[1], and the train/test/validation splits were manually organized based on predefined or custom sampling strategies to suit the training objectives and disk space limitations.
To prepare the dataset for efficient loading and training, `Wave-U-Net`'s `dataset.py` pipeline was used to preprocess and convert the raw audio files into structured `.hdf5` files. During this step, the model loaded each stereo mixture (`mix`) along with the aligned clean stems (`bass`, `drums`, `vocals`, and `other`), and stored them as paired inputs and targets. The processed `.hdf5` files capture time-domain waveforms rather than spectrograms, enabling end-to-end training directly on audio samples.

- **Each `.hdf5` group contains:**
  - `inputs`: raw waveform of the stereo mixture
  - `targets`: concatenated waveforms of the clean source tracks

[1] `https://sigsep.github.io/datasets/musdb.html`

– `length` and `target_length`: metadata to ensure correct cropping and segmentation

The preprocessing pipeline also computed output-aligned training windows, enabling efficient random sampling of input segments during training. Optional audio augmentation (e.g., `random_amplify`) was applied during training to enhance generalization.
To generate the `.hdf5` datasets, the user specified:

- Dataset root directory: `--dataset_dir`
- HDF5 output path: `--hdf_dir`
- Target instruments: `--instruments`
- Sample rate, input/output lengths, and stride size

By using the Wave-U-Net data loader and preprocessing script, the model was able to handle long input sequences, capture phase information, and train effectively on the full waveform domain. This preprocessing step played a critical role in enabling fast and consistent I/O for large-scale model training and accurate source separation.

Training Setup

We conducted experiments on two GPU platforms: NVIDIA Tesla V100 (32 GB VRAM, Volta architecture, 300 W TDP). NVIDIA Tesla P100 (16 GB VRAM, Pascal architecture, 250 W TDP). The Wave-U-Net model was trained on the MUSDB18 training set (which consists of audio tracks for source separation) for 3 epochs in each experiment. We evaluated two batch sizes: Batch Size 1 (one audio example per training step) and Batch Size 4 (four examples per step). The dataset and model were identical in all runs, ensuring a fair comparison. We recorded the following metrics during training: Throughput (samples/sec) and Epoch Duration (sec): how many training examples are processed per second, and how long each epoch takes to complete. GPU Memory Usage (MiB) and Memory Utilization (%): the absolute GPU memory consumption and the percentage of total VRAM used. GPU Utilization (%): the percentage of time the GPU's compute units were busy (a measure of how fully the GPU is being utilized). Power Consumption (W): instantaneous power draw of the GPU during training. Temperature (°C): GPU core temperature during training. All runs were monitored with NVIDIA's profiling tools (e.g., nvidia-smi in persistence mode logging at regular intervals). The metrics were logged over time as training progressed through the 3 epochs. Below, we present and discuss the results, with plotted graphs for each metric on each GPU (batch size 1 vs. 4) and detailed comparisons of the advantages and disadvantages observed for each configuration.

GPU Utilization Summary

Throughput and Epoch Duration

Training Wave-U-Net on both the V100 and P100 GPUs showed that increasing the batch size from 1 to 4 significantly improved throughput and reduced epoch time. On the V100, batch size 4 achieved a throughput of up to 9.76 samples/sec, nearly doubling the speed compared to batch size 1 (5.41 samples/sec). Similarly, the P100 saw throughput rise from 5.20 to 5.23 samples/sec. While both GPUs benefited from larger batches, the V100 consistently outperformed the P100 across all runs. Notably, V100 with batch size 4 completed training in nearly half the time of its batch size 1 configuration, demonstrating its superior parallel processing capabilities and architectural efficiency.

GPU Memory Usage and Utilization

Larger batch sizes led to significantly higher memory usage on both GPUs. On the P100, batch size 4 consumed approximately 3450MiB (21% of 16GB), while batch size 1 used around 1930MiB (12%). On the V100, which has 32GB of memory, the same batch sizes resulted in similar absolute usage (3450MiB for batch 4 and 1930MiB for batch 1), translating to only 10.5% and 5.9% of total VRAM, respectively. These results show that batch size 4 utilized nearly double the memory of batch size 1 but remained well within the capacity of both GPUs. While the P100 operated closer to its memory ceiling, the V100 had substantial headroom for larger batch sizes or more complex models. Thus, memory was not a bottleneck in either case, but the V100 offers more flexibility for scaling.

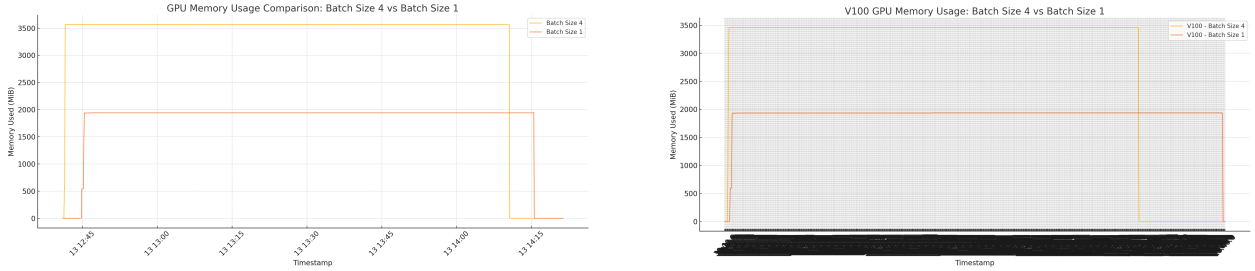


Figure 6: GPU Memory Usage: P100 vs V100 (Batch Size 1 vs 4)

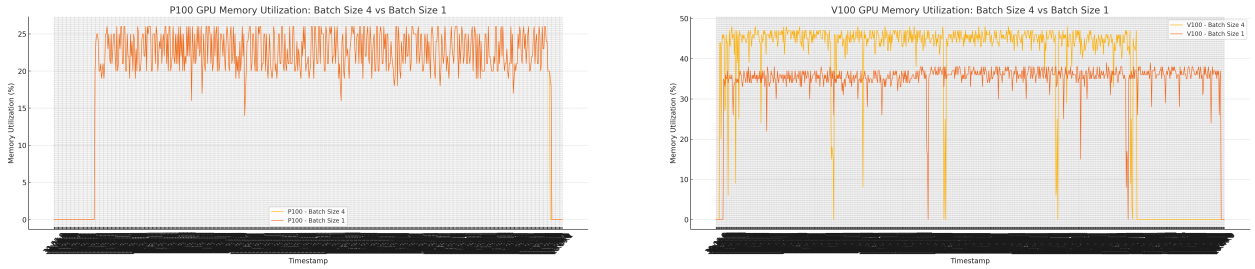


Figure 7: GPU Memory Utilization (% of VRAM): P100 vs V100

GPU Compute Utilization

GPU utilization was consistently higher with batch size 4 compared to batch size 1 on both the P100 and V100. On the P100, batch size 4 maintained GPU usage near 90–100% for most of the training period, while batch size 1 showed lower and more erratic utilization, often dipping to 50% or lower due to frequent idle periods. Similarly, on the V100, batch size 4 resulted in more stable and higher utilization (typically 95–100%), whereas batch size 1 had more frequent dips, although its active periods still saw high utilization (85–90%). Overall, batch size 4 was more effective at keeping both GPUs fully occupied, reducing idle time and improving training efficiency.

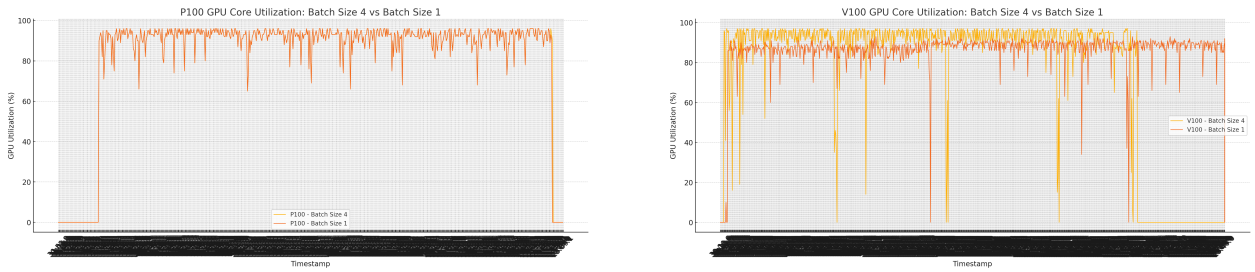


Figure 8: GPU Core Utilization: P100 vs V100 (Batch Size 1 vs 4)

Power Consumption

Power usage was consistently higher with batch size 4, reflecting greater GPU utilization. On the P100, batch 4 peaked around 230 W, while batch 1 stayed near 150 W. Despite consuming more power, batch 4 completed training significantly faster, making it potentially more energy-efficient overall. The V100 exhibited similar trends but with higher absolute values: batch 4 reached near its TDP limit (290–300 W), whereas batch 1 ranged between 150–200 W. Again, batch 4 finished faster and used its power more efficiently despite a higher draw. In summary, batch size 4 results in higher power usage but more effective GPU utilization and shorter training times, making it preferable for performance-oriented training.

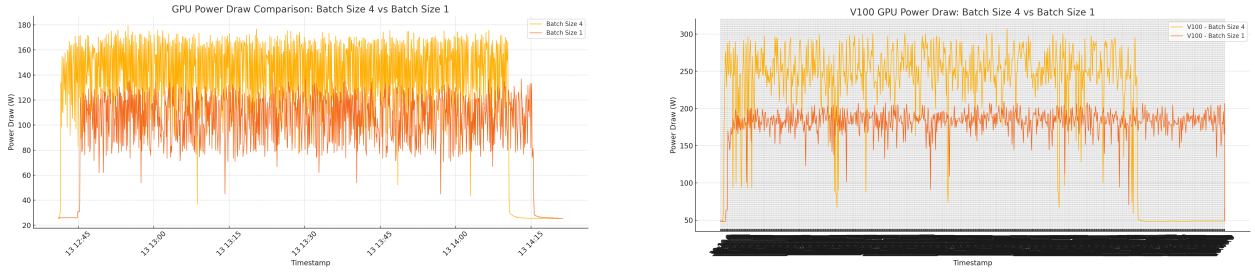


Figure 9: GPU Power Consumption: P100 vs V100 (Batch Size 1 vs 4)

Temperature

Temperature measurements revealed that larger batch sizes resulted in higher GPU temperatures due to increased utilization and power draw. On the P100, batch size 4 stabilized around 60–62°C, while batch size 1 remained cooler near 50–55°C. The V100 showed a similar pattern, with batch 4 peaking at 68°C and batch 1 peaking around 60°C. Despite running hotter, both GPUs stayed well below thermal throttling thresholds, and the higher temperature of batch 4 corresponds with its faster and more efficient training performance. Cooling systems in both setups managed the thermal load effectively.

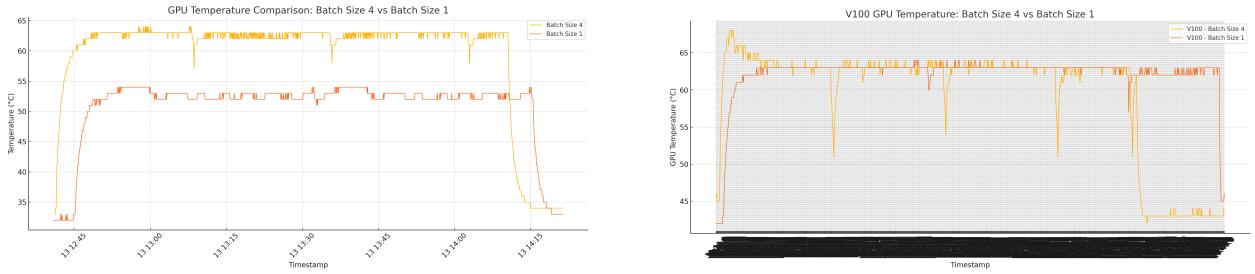


Figure 10: GPU Temperature Trends: P100 vs V100 (Batch Size 1 vs 4)

This comprehensive evaluation of NVIDIA V100 and P100 GPUs for training the Wave-U-Net model on the MUSDB18 dataset reveals the following key findings:

- **Throughput and Speed:**
  - V100 consistently outperformed P100 across all batch sizes.
  - With batch size 4, the V100 completed training in less than half the time required by the P100.
  - Even with batch size 1, V100 was significantly faster than P100.
- **Batch Size Scaling:**
  - Increasing batch size from 1 to 4 improved throughput on both GPUs.
  - V100 saw better scaling benefits due to superior parallel processing.
  - Batch size 1 led to underutilization and longer training times.
- **Memory Usage:**
  - Batch 4 used approximately 3.4GB; batch 1 used 1.9GB on both GPUs.
  - V100 used only 10.5% of its memory; P100 used about 21%.
  - Memory was not a limiting factor in this setup; V100 offers more headroom for future scaling.
- **GPU Compute Utilization:**
  - Batch 4 achieved nearly 100% GPU utilization on both V100 and P100.
  - Batch 1 showed lower and more erratic utilization, especially on the P100.
  - V100 maintained higher utilization even with smaller batches due to its architectural efficiency.
- **Power Consumption and Efficiency:**
  - V100 drew more power under load (250–300W) compared to P100 (150–230W).
  - Batch 4 pushed both GPUs to near-maximum power draw.
  - Despite higher instantaneous power, V100 completed tasks faster, making it more energy-efficient.
  - P100 consumed less power per moment but ran longer, negating energy savings.

- **Temperature and Thermal Behavior:**
  - V100 ran hotter (68°C) than P100 (60°C) with batch size 4.
  - Batch 1 resulted in lower temperatures on both GPUs (10°C cooler).
  - No thermal throttling was observed; both GPUs operated within safe thermal limits.
- **Configuration Summary:**
  - **Best:** V100 with batch size 4 – highest throughput, excellent utilization, memory/power headroom.
  - **Good:** V100 with batch size 1 – still faster than all P100 configurations but not fully utilized.
  - **Better:** P100 with batch size 4 – improved over batch 1, good utilization, moderate speed.
  - **Worst:** P100 with batch size 1 – slowest, underutilized, least efficient.
- **Practical Recommendations:**
  - Use the largest batch size that fits in memory to maximize performance.
  - Prefer V100 for faster, more energy-efficient training, especially for long or large-scale jobs.
  - P100 is adequate for small-scale or resource-constrained environments but will require longer runtimes.
  - Batch tuning is a simple yet effective optimization lever.

Highlights

As the field of audio enhancement continues to grow, one of the most exciting future directions is the use of transfer learning. This approach allows machine learning models like DeepFilterNet3 and Wave-U-Net, which are trained on large datasets such as MS-DNS and MS-SNSD, to be adapted efficiently for more specific and practical use cases. Instead of training models from scratch, transfer learning helps in fine-tuning pre-trained models using smaller, customized datasets that reflect particular environments like offices, vehicles, or classrooms. This strategy brings several benefits. It saves time and computational resources during training and improves model performance in new or unseen noise conditions. For example, a model originally trained to remove general background noise can be further fine-tuned to work well in specific settings like industrial noise or classroom chatter without needing a large new dataset. Both DeepFilterNet3 and Wave-U-Net can benefit from this process, making them more robust, flexible, and personalized for real-world applications. In future research, it will be important to explore techniques such as which layers to freeze, how to adjust learning rates for different parts of the network, and what data augmentation methods help the most. Researchers can also look at how well models trained on one type of audio problem can be reused for another — for instance, using a model trained for speech enhancement to help with music separation. This can help build more universal and reliable audio enhancement systems, with Wave-U-Net and DeepFilterNet3 leading the way.

Future Direction

The comparative analysis of GPU utilization during the training of DeepFilterNet and Wave-U-Net reveals how underlying GPU architecture significantly impacts the practicality of edge learning. When subjected to varying batch sizes, larger batch sizes, such as 2 or 4, generally yield higher training throughput, better GPU occupancy, and more stable power and thermal characteristics, particularly on high-performance data center GPUs like the NVIDIA V100. However, these benefits do not easily translate to edge hardware due to tighter constraints on memory, thermal dissipation, and power budget. On such devices, batch size 1 is often preferred for its low memory footprint, but our analysis shows that this configuration can result in higher average power draw and increased thermal stress, which over time could throttle performance or reduce hardware lifespan. Wave-U-Net, with its deep multi-scale architecture and wide receptive fields, tends to be more memory- and compute-intensive compared to DeepFilterNet, making it especially challenging to deploy on constrained devices without architectural modifications. To enable effective on-device training and inference, modern mobile SoCs have started integrating specialized AI hardware—such as Apple's A17 Pro Neural Engine, Qualcomm's Adreno GPU on Snapdragon 8 Gen 3, and ARM's Mali-G715—which are capable of mixed-precision computation and energy-efficient inference. Embedded platforms like NVIDIA Jetson Orin Nano and Xavier NX offer CUDA support and strike a balance between compute capability and deployment flexibility, making them suitable for lightweight training and real-time execution of models like Wave-U-Net. To meet the demands of edge learning, model adaptations such as parameter pruning, quantization, and architectural simplification can be employed. Furthermore, training strategies like continual learning and incremental fine-tuning allow models to evolve in response to local data while minimizing resource usage. The findings from V100 and P100 training experiments serve not only as a benchmark for GPU behavior but also as a foundation for designing scalable, energy-aware training and deployment strategies for real-time audio processing at the edge.

Conclusion

This study presents a detailed evaluation of GPU performance during the training of two deep learning-based noise suppression models—WaveUNet and DeepFilterNet. DeepFilterNet was trained on the MS-DNS and MS-SNSD speech enhancement datasets, while WaveUNet used the MUSDB18 dataset for music source separation. Experiments were conducted on NVIDIA P100 and V100 GPUs using batch sizes of 1 and 2 to assess efficiency, scalability, and hardware suitability. Metrics such as memory usage, GPU and memory clock speeds, power consumption, thermal behavior, and throughput were analyzed. On the P100, batch size 1 yielded lower memory usage but caused power fluctuations and reduced throughput, while batch size 2 strained the GPU's ability to maintain consistent utilization and clock speeds. The V100, by contrast, delivered higher throughput, better memory efficiency, and more stable thermal and power profiles—particularly with batch size 2—thanks to its superior architecture and support for mixed-precision computation. GPU utilization plots showed the V100 maintained steady workloads, whereas the P100 displayed fragmented activity. These insights are critical for edge and adaptive learning scenarios, where efficiency and consistency are vital. While the V100 is ideal for high-performance training, the P100 offers a cost-effective alternative for smaller-scale tasks. The findings help guide hardware and training configuration choices for deploying WaveUNet and DeepFilterNet across varied audio enhancement applications.

References

[1] Simon Dixon Daniel Stoller Sebastian Ewert. "Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation". In: *https://arxiv.org/abs/1806.03185* (2018). URL: https://arxiv.org/abs/1806.03185.

[2] Yu Luo and Lina Pu. "EC-ANC: Edge Case-Enhanced Active Noise Cancellation for True Wireless Stereo Earbuds". In: *IEEE/ACM Trans. Audio, Speech and Lang. Proc.* 30 (Apr. 2022), pp. 1436–1447. ISSN: 2329-9290. DOI: 10.1109/TASLP.2022.3164211. URL: https://doi.org/10.1109/TASLP.2022.3164211.

[3] Varun Patkar et al. "Audio Source Separation using Wave-U-Net with Spectral Loss". In: *2023 International Conference on Communication System, Computing and IT Applications (CSCITA)*. 2023, pp. 101–106. DOI: 10.1109/CSCITA55725.2023.10104853.

[4] Azeddine Wahbi et al. "Embedded Acoustic Noise Cancellation (ANC) Performances Analysis for Voice Communications using Linear Adaptive Approaches". In: *Proceedings of the 6th International Conference on Networking, Intelligent Systems & Security*. NISS '23. Larache, Morocco: Association for Computing Machinery, 2023. ISBN: 9798400700194. DOI: 10.1145/3607720.3607741. URL: https://doi.org/10.1145/3607720.3607741.

[5] Shilin Wang et al. "Improving low-complexity and real-time DeepFilterNet2 for personalized speech enhancement". In: *Int. J. Speech Technol.* 27.2 (Apr. 2024), pp. 299–306. ISSN: 1381-2416. DOI: 10.1007/s10772-024-10101-z. URL: https://doi.org/10.1007/s10772-024-10101-z.

[6] Yu Zhang, Jiangtao Wen, and Yuxing Han. "Adaptive Learning Based Active Noise Cancellation". In: *Proceedings of the 3rd International Conference on Multimedia and Image Processing*. ICMIP '18. Guiyang, China: Association for Computing Machinery, 2018, pp. 41–45. ISBN: 9781450364683. DOI: 10.1145/3195588.3195591. URL: https://doi.org/10.1145/3195588.3195591.